Week 2 Design Retrospective — Resource Planning Ledger

CSCI-P532 Object Oriented Software Development, Spring 2026

Change 1 · New Action States and Guards (State Pattern)

Added	Modified	Untouched (notable)

`PendingApprovalState`
`ReopenedState`
`ReversalLedgerEntryGenerator`
`ActionApprovalController`
`ActionApprovalManager`
	LOGIC
	`ProposedState` — implement() throws; submitForApproval() added
	`CompletedState` — reopen() transition added
	
	MECHANICAL (ENUM EXTENSION ONLY)
	`ActionStatus` — new constants PENDING_APPROVAL, REOPENED
	`ActionState` — default-throw stubs for new event methods
	`Plan` — derived-status switch updated for new values
		`AbstractLedgerEntryGenerator`
		`ConsumableLedgerEntryGenerator`
		`InProgressState`
		`SuspendedState`
		`AbandonedState`
		`ActionContext`
		`ActionContextCallback`

Analysis: Largely landed where the design predicted — logic changes were confined to two state classes. The three mechanical updates (`ActionStatus`, `ActionState`, `Plan`) are unavoidable enum/interface extension costs, not design failures. Redesign: replace the `ActionState` *interface* with an *abstract base class* (default-throw implementations built in) and add a `statusCategory()` method to `ActionStatus` — both changes would reduce mechanical churn to zero for any future state addition.

Change 2 · Asset Ledger Entry Generator (Template Method Pattern)

Added	Modified	Untouched (notable)

`AssetLedgerEntryGenerator`	**LOGIC — ONE LINE**
	`ActionManager` — onComplete() calls assetLedgerEntryGenerator .generateEntries() after consumable posting
		`AbstractLedgerEntryGenerator`
		`ConsumableLedgerEntryGenerator`
		`LedgerEngine` `LedgerManager`
		All repositories

Analysis: The Template Method pattern absorbed this change almost perfectly — zero changes to the base class or any other subclass. One friction point: `findOrCreateUsageAccount()` was `private`, forcing the new subclass to inline the three-line account-lookup logic. Declaring it `protected` in Week 1 would have eliminated that duplication; alternatively, extracting it to a shared `AccountResolver` @Component would make it injectable by any generator.

Change 3 · Filtered and Lazy Iterators (Iterator Pattern)

Added	Modified	Untouched (notable)

`FilteredPlanIterator`
`LazySubtreeIterator`
`PlanReportManager`
`PlanQueryController`
	ROUTING ONLY — NO LOGIC CHANGE
	`PlanController` — optional `?status=` param; delegates to `PlanReportManager`
		`DepthFirstPlanIterator`
		`Plan` `ProposedAction`
		`PlanManager` `ReportManager`
		All repositories

Analysis: Landed cleanly. The predicate-decorator design of `FilteredPlanIterator` kept all filtering logic out of the Composite classes entirely. The `PlanController` touch was purely additive routing; a second dedicated controller endpoint from the start (`PlanQueryController`) would have left even `PlanController` untouched.

Change 4 · Visitor-Based Plan Metrics (Visitor + Composite Pattern)

Added	Modified	Untouched (notable)

`PlanNodeVisitor`
`CompletionRatioVisitor`
`ResourceCostVisitor`
`RiskScoreVisitor`
`PlanMetricsService`
`PlanMetricsController`
	None. `accept(PlanNodeVisitor)` was already wired in Week 1 on `PlanNode`, `Plan`, and `ProposedAction` per the design spec.	`Plan` `ProposedAction` `PlanNode`
		All managers All repositories
		All existing controllers

Analysis: The cleanest change of all four — zero modifications to any existing class. This confirms that wiring `accept()` in Week 1 was the correct forward-looking decision; the Visitor extended the Composite with no blast radius whatsoever.

Overall Assessment

Across all four changes: **2 real logic** modifications (`ProposedState`, `CompletedState`), **1 single-line extension** (`ActionManager`), **1 routing-only touch** (`PlanController`), and **3 mechanical updates** driven solely by new enum values (`ActionStatus`, `ActionState`, `Plan`). The design succeeded wherever extension points were made explicit in advance — Template Method hooks in the ledger hierarchy and Visitor `accept()` wired into the Composite — and showed friction wherever they were left implicit: closed interface method lists, private base-class helpers, and exhaustive enum switches. The primary improvements would be (1) replacing the `ActionState` interface with an abstract base class to eliminate interface churn on every new state, and (2) adding a `statusCategory()` method to `ActionStatus` to insulate `Plan`'s rollup logic from future enum growth.